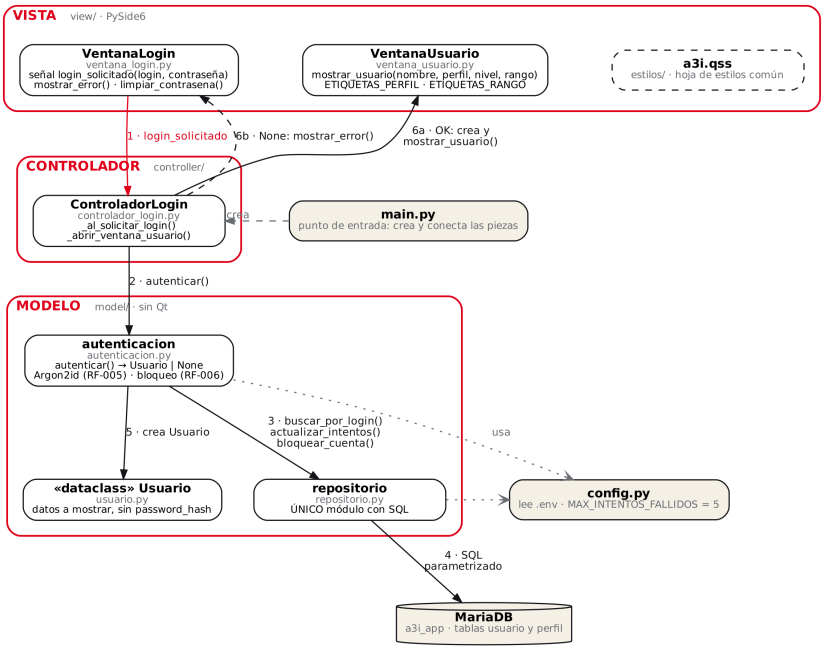
Diseño de la estructura MVC

Estructura de la aplicación de escritorio (PySide6) para el Sprint 1: login conectado a MariaDB y ventana con los datos del usuario (RF-008). Las carpetas siguen el patrón Modelo–Vista–Controlador; la vista y el modelo nunca se importan entre sí.

1. Diagrama de capas



Flechas continuas: flujo del login (1–6). Discontinuas: arranque y caso de error. Punteadas: lectura de configuración.

2. Reglas de cada capa

CAPA	QUÉ HACE	QUÉ TIENE PROHIBIDO
Modelo `model/`	Datos y lógica: consultas, verificación de la contraseña y bloqueo (RF-006).	Importar PySide6 o mostrar nada en pantalla.
Vista `view/`	Pintar widgets y emitir señales, p. ej. `login_solicitado(login, contraseña)`.	Importar el modelo o tomar decisiones.
Controlador `controller/`	Recibir la señal, llamar al modelo y decidir qué vista mostrar.	Hacer SQL o construir widgets.

3. Decisiones de diseño

- **Todo el SQL en `repositorio.py`**, siempre con consultas parametrizadas (RF-014, RNF-04). Cuando exista la API (RF-004), solo se sustituye este módulo por un cliente HTTP.
- **`autenticar()` devuelve un `Usuario` o `None`.** El hash nunca sale del modelo y el mensaje de error es siempre el mismo (RNF-19).
- **La vista no conoce `Usuario`:** el controlador le pasa valores sueltos y la vista los traduce a etiquetas (`scientist` → «Científicos»).
- **El controlador guarda `_ventana_usuario` como atributo**, para que Qt no libere la ventana al abrirla.
- **Configuración en `config.py`** (lee `.env` con `python-dotenv`): credenciales de la BD y `MAX_INTENTOS_FALLIDOS = 5`.
- **Deuda técnica:** en el Sprint 1 la aplicación se conecta directamente a MariaDB con PyMySQL, no a través de la API.